

AI-PBMS FAULT DETECTION DESIGN REPORT

Analysis of the Existing Machine Learning Fault Classification
and Verification Logic in the BMS Project

CATERPILLAR TECH CHALLENGE

Project Architecture Documentation

Date: June 2026

A. Existing ML Pipeline Summary

The battery management system (BMS) implements a hybrid physics-informed machine learning pipeline for real-time anomaly detection and safety shutdown. The design uses both structural physics knowledge and tabular machine learning to classify fault states. Key pipeline blocks include:

- * **Data Ingestion:** Receives raw packet streams over Bluetooth Low Energy (BLE) from JBD/Xiaoxiang or Daly BMS hardware, or via a local sensor simulator script, containing cell voltages, temperatures, pack voltage, current, and State of Charge (SOC).
- * **Validation & Filter:** Processes raw streams through range validation limits. Invalid or corrupt telemetry rows are immediately flagged and excluded from the active inference pipeline.
- * **Feature Engineering:** Extracts physics-informed derivatives, rolling statistics, cell-wise differentials, and operating modes on a sliding 60-second telemetry lookback window.
- * **XGBoost Inference:** Applies a MinMaxScaler and feeds the aligned 50 engineered features of the latest row into a trained XGBoost Classifier to output predictions and class probabilities.
- * **Decision Engine:** Applies specific confidence-based thresholds (40%-50%) on predicted probabilities to issue recommended alerts and corrective actions displayed on the Web Dashboard.
- * **Incremental Learning:** Uses background scheduler loops to label accumulated telemetry rows via hardcoded threshold rules, appends them to the primary dataset, and retrains the model in the background.

B. Files Involved in the ML System

The ML fault detection system spans several files in the workspace:

- * **bms_xgboost_model.json:** Trained model weights containing the tree ensemble definition for the 6-class XGBoost classifier.
- * **bms_scaler.joblib:** Fitted MinMaxScaler object used to normalize the raw 50 engineered features before model inference.
- * **feature_columns.json:** JSON list defining the exact sequence and names of the 50 features fed into the model.
- * **label_map.json:** JSON map mapping numerical classes (0-5) to string fault names.
- * **predict_fault.py:** Core inference module. Performs feature calculations, operating mode classification, data scaling, and calls the XGBoost model to output recommended actions.
- * **train_xgboost.py:** The primary model training script that reads the spreadsheet dataset, engineers features, fits the MinMaxScaler, trains XGBoost, and generates classification reports.
- * **incremental_train.py:** Background worker triggered by the dashboard backend every 10 minutes. It labels newly accumulated data using threshold rules and retrains the model.
- * **generate_synthetic_faults.py:** Uses baseline normal battery datasets and injects synthetic faults (drifts, sags, RC rises) using circuit models to build the training set.
- * **bms_dashboard_backend.py:** Flask server running on the Pi. It serves the dashboard frontend, handles BLE/Simulator API telemetry inputs, runs data validation, maintains the sliding buffer, and calls inference.
- * **bms_bluetooth_gateway.py:** BMS BLE driver that polls Daly or JBD BMS parameters and forwards them to the Flask server.
- * **laptop_live_logger.py:** Desktop-side receiver script that logs raw and filtered CSV files and launches the local browser dashboard.

C. Input Features Used by the Model

The model receives an input matrix of size [1, 50] representing the 50 physical features extracted from the 60-row lookback window at the current timestamp. Features are classified as follows:

- * **Raw Sensor Values:** voltage (pack voltage), current (pack current), temperature (pack temperature), soc (State of Charge %), ntc1-4 (four temperature thermistors), cell_v1 to cell_v8 (individual cell voltages).
- * **Derived Statistics:** delta_v (maximum cell voltage minus minimum cell voltage), cell_max (max cell voltage), cell_min (min cell voltage), cell_mean (mean cell voltage), cell_std (standard deviation of cell voltages), cell_range (equivalent to delta_v).
- * **Operating Mode:** mode_ACCEL, mode_CRUISE, mode_DECEL, mode_IDLE. One-hot encoded operating conditions derived from current and voltage change rates to provide load context.
- * **Time-Series Derivatives:** dv_dt (rate of change of delta_v), dtemp_dt (temperature change rate), dsoc_dt (SOC change rate), dvoltage_dt (pack voltage change rate).
- * **Rolling Statistics:** rolling_mean_voltage, rolling_std_voltage, rolling_mean_temperature, rolling_std_temperature (calculated using a rolling 10-sample window).
- * **Cell-wise Drop Rates:** cell_drop_rate_1 to cell_drop_rate_8 (captures the magnitude of voltage drops per cell. Computed as the absolute value of diff() clipped to negative-only, representing sag rates).
- * **Cell-wise Rise Rates:** cell_rise_rate_1 to cell_rise_rate_8 (captures the magnitude of voltage recovery per cell. Computed as diff() clipped to positive-only).

D. Data Filtering Currently Implemented

Data validation is executed at the Flask gateway (is_valid_telemetry in bms_dashboard_backend.py). Data that fails is rejected before it reaches the inference engine.

- * **Invalid Timestamps:** Not Implemented. If missing from the request, the backend automatically generates a timestamp using local server time.
- * **Duplicate Timestamps:** Not Implemented in backend. (Only laptop logger performs deduplication before writing to local CSVs).
- * **Missing / NaN Values:** Partially Implemented. Telemetry properties are cast to floats. If casting raises an exception, the row is discarded. Missing cells default to 3.2V; missing NTCs default to pack temperature.
- * **Zero Voltage Value:** Implemented. Handled by physical pack limit of 15.0V $\leq$ voltage $\leq$ 40.0V. A zero-voltage row is discarded.
- * **Zero Temperature Value:** Not Filtered. 0.0 C is treated as valid temperature since the acceptable range is -40.0 C to 120.0 C.
- * **Zero Current Value:** Not Filtered. 0.0 A is valid (Idle mode).
- * **Repeated / Stuck Sensor:** Not Implemented. There is no check to detect if a sensor value is frozen.
- * **Out-of-Range Parameters:** Implemented. Enforces: 15.0V $\leq$ voltage $\leq$ 40.0V, -120.0A $\leq$ current $\leq$ 120.0A, -40.0C $\leq$ temperature $\leq$ 120.0C, 0.0% $\leq$ soc $\leq$ 100.0%, and cell_v1-8 must each be 2.0V $\leq$ cell_v $\leq$ 4.6V.
- * **Noisy Spikes:** Not Implemented. No outlier removal or filtering is applied to noisy signals.
- * **Mismatched Features:** Implemented. Aligned in predict_fault.py using feature_columns.json columns. If a feature column is missing or mismatched, it raises a KeyError/ValueError.
- * **Incomplete Rows:** Implemented. If required base columns are missing, float casting or default checks fail, and the row is discarded.

Failing Row Action: If a row fails any rule in `is_valid_telemetry`, the backend skips writing to `bms_telemetry_filtered.csv`, returns status: `corrupt_discarded`, and excludes the row from the inference queue and retraining buffer. (Note: The raw row is logged to `bms_telemetry_raw.csv` before validation).

E. Feature Extraction Currently Implemented

Before prediction, features are calculated dynamically on the 60-row sliding window (engineer_features in predict_fault.py):

- * **max cell voltage:** Calculated using `df[cell_cols].max(axis=1)`. Used by ML model.
- * **min cell voltage:** Calculated using `df[cell_cols].min(axis=1)`. Used by ML model.
- * **average cell voltage:** Calculated using `df[cell_cols].mean(axis=1)`. Used by ML model.
- * **cell voltage spread:** Calculated as `delta_v = cell_max - cell_min` and `cell_range`. Used by ML model.
- * **cell standard deviation:** Calculated using `df[cell_cols].std(axis=1)`. Used by ML model.
- * **voltage sag rate:** Not calculated explicitly, but `cell_drop_rate_i` represents individual cell drop rates. Used by ML model.
- * **voltage recovery rate:** Not calculated explicitly, but `cell_rise_rate_i` represents individual cell rise rates. Used by ML model.
- * **dV/dt:** Calculated as `dv_dt = delta_v.diff()` and `dvoltage_dt = voltage.diff()`. Used by ML model.
- * **dT/dt:** Calculated as `dtemp_dt = temperature.diff()`. Used by ML model.
- * **current trend:** Calculated as `dl_dt = current.diff()`. Used to determine operating mode, not fed directly.
- * **SOC trend:** Calculated as `dsoc_dt = soc.diff()`. Used by ML model.
- * **temperature difference:** Not calculated. (NTC1-4 are fed directly, but cross-NTC differences are not engineered).
- * **thermal rise rate:** Calculated as `dtemp_dt` (derivative of pack temperature). Used by ML model.
- * **operating mode:** Calculated via `current`, `dl_dt`, and `dV_dt`. Classified into IDLE, ACCEL, DECEL, or CRUISE. One-hot encoded and used by ML model and displayed.

F. Fault-Wise Classification Logic

The model classifies battery status into one of 6 classes. Below is the technical breakdown of each fault:

- * **Normal (Class 0):** Influenced by flat temperature, stable cells, and `delta_v < 0.070V`. Detected by ML. Evaluates latest sample. Alert condition: highest probability is Normal. Action: 'Normal operation. No action required.'
- * **Cell Imbalance (Class 1):** Influenced by `delta_v`, `cell_range`, `cell_std`, and `mode_IDLE`. Labeled in retraining if `delta_v > 0.080V` and `current < 0.10A`. Detected by ML. Alert condition: Imbalance probability > 0.40. Actions: 'Start balancing' (if IDLE) or 'Schedule balancing at next idle cycle'.
- * **Weak Cell (Class 2):** Influenced by `cell_drop_rate_i`, `cell_std`, `mode_ACCEL`, and `current`. Labeled in retraining if `delta_v > 0.070V` and `current < -0.10A`. Detected by ML. Alert condition: Weak Cell probability > 0.50. Action: 'Reduce allowable discharge current' (if ACCEL).
- * **Overvoltage Risk (Class 3):** Influenced by `cell_max`, `cell_v1-8`, rolling mean voltage, and `mode_DECEL`. Labeled in retraining if `max(cell_v) > 4.18V` or `pack voltage > 33.2V` or `(max(cell) > 4.15V and current > 0.05A)`. Detected by ML. Alert condition: Overvoltage probability > 0.40. Action: 'Stop charging' (if DECEL) or 'Monitor pack voltage; reduce charge rate'.
- * **Undervoltage Risk (Class 4):** Influenced by `cell_min`, `cell_v1-8`, rolling mean voltage, and `mode_ACCEL/CRUISE`. Labeled in retraining if `min(cell_v) < 2.85V` or `pack voltage < 22.8V` or `(min(cell) < 2.90V and current < -0.05A)`. Detected by ML. Alert condition: Undervoltage probability > 0.40. Action: 'Reduce load/Stop discharge immediately'.
- * **Overtemperature Risk (Class 5):** Influenced by `temperature`, `ntc1-4`, and `dtemp_dt`. Labeled in retraining if `temperature > 45.0 C`. Detected by ML. Alert condition: Overtemperature probability > 0.40. Action: 'Cool battery pack/Stop operation immediately'.

Note: Softmax outputs class probabilities. The predicted fault is `argmax(probs)`, and decision alerts use probability thresholds

(40%-50%). No physical hardware corrective action is triggered; the dashboard only displays the recommended action string.

G. Training Process Explanation

The model is a tabular gradient boosted tree classifier trained and validated locally using standard scikit-learn metrics.

- * **Dataset:** augmented_telemetry_dataset.xlsx, which concatenates normal operating cycles with synthetically generated fault blocks (drifts, sags, temperature rises).
- * **Dataset Size:** 2,756 rows (Train size = 2,204 rows, Test size = 552 rows, using a stratified 80/20 train/validation split).
- * **Labeling Method:** The initial training set is labeled synthetically based on equivalent-circuit/RC model injections (generate_synthetic_faults.py). The incremental retraining script labels incoming data using hardcoded thresholds.
- * **XGBoost Configuration:** n_estimators=120, max_depth=4, learning_rate=0.08, subsample=0.85, colsample_bytree=0.85, random_state=42. Optimized for execution speed and portability.
- * **Validation Metrics:** The model achieves an overall validation accuracy of 98.55%. Class-specific F1-scores are: Normal (0.99), Cell Imbalance (0.97), Weak Cell (1.00), Overvoltage Risk (0.96), and Undervoltage Risk (0.97).
- * **Overfitting Risks:** Extremely High. Because the faults were synthetically generated using simple deterministic mathematical rules (e.g., subtracting a drift from cell_v3), the XGBoost model likely memorizes these specific synthetic profiles rather than generalized physical cell degradation paths.

H. Real-Time Inference Flow

Real-time prediction executes sequentially as follows:

1. Telemetry Reception: The gateway client POSTs raw telemetry JSON to /api/telemetry.
2. Raw Logging: Flask writes the raw row to bms_telemetry_raw.csv.
3. Validation Check: Evaluates ranges in is_valid_telemetry. If invalid, the row is discarded. If valid, it writes to bms_telemetry_filtered.csv and is added to the in-memory learning queue.
4. Sliding Buffer Update: Valid telemetry is enqueued in telemetry_buffer (size 60). If length matches 60, inference begins.
5. Feature Engineering: engineers_features lowercases columns, calculates statistics (max, min, std, range), computes derivatives (dv_dt, dtemp_dt, etc.), and identifies operating modes.
6. Feature Alignment & Scaling: Aligns features to feature_columns.json order and normalizes features using bms_scaler.joblib.
7. Model Inference: Extracts the latest row (index 59) and runs prediction through the cached XGBoost classifier.
8. Decision Engine: Maps probabilities to label_map.json and checks action thresholds to select the Recommended Action.
9. Dashboard Render: Returns the JSON payload to gateway loggers and live_dashboard_v3.html to display alerts in real-time.

I. Limitations and Design Weaknesses

The current design contains several critical limitations that affect real-world reliability:

1. Lack of Noise Filtering: No signal cleaning (like Kalman or low-pass filtering) is applied. Sensor noise could cause false fault alerts.
2. Circular Logic: Retraining relies on threshold-labeled data. The machine learning model is trained to mimic hardcoded thresholds. This makes the ML model redundant, as the threshold logic could be used directly.
3. Synthetic Data Overfitting: The training set is small (under 3,000 rows) and consists of simple synthetic fault profiles, creating high risk that the model will fail on real, complex battery cells.
4. No Alarm Persistence: Alerts are triggered row-by-row on the latest sample. There is no hysteresis or persistence check (e.g., verifying a fault remains active for 3 seconds) to prevent nuisance alarms.
5. Simplistic Thermal Hotspot Logic: The system checks absolute pack temperature. It does not compare NTC sensors against

each other to identify local physical hotspots.

6. No Physical Recovery Verification: Cell rise rates are calculated, but the model lacks physical relaxation logic to separate internal resistance growth from capacity loss.

J. What Can Be Confidently Told to Judges

- * **Explainable Tabular ML:** The system replaces a black-box deep learning model with an explainable XGBoost architecture, which uses clear decision boundaries based on physical features.
- * **Edge-Deployable Efficiency:** The model runs in under 1ms on a Raspberry Pi CPU, avoiding thermal throttling and leaving CPU cycles free for BLE polling.
- * **Physics-Informed Engineering:** We engineer 50 battery-specific features (like cell-wise drop rates, voltage spread, standard deviation, and derivatives) rather than feeding raw values to a model.
- * **Context-Aware Decision Engine:** Operating modes (Acceleration, Cruise, Deceleration, Idle) are detected dynamically to provide context for warnings (e.g., reducing discharge current only during acceleration).
- * **Automated Retraining Loop:** An automated background worker enables the system to continuously label, store, and retrain the model on live telemetry.

K. What Should NOT Be Claimed to Judges

- * **Predictive Forecasting:** Do not claim that the model forecasts future faults. It is a classifier that detects anomalies within the active 60-row window.
- * **Real Cell Degradation Training:** Do not claim that the model was trained on thousands of hours of real degraded battery hardware. Training used baseline telemetry augmented with synthetic faults.
- * **Deep Learning / Transformers:** Do not claim that the live system runs on the Transformer model. The Transformer model was deprecated due to lower accuracy (86.32% vs. 98.55%) and high computational load.
- * **Active Hardware Shutdowns:** Do not claim that the system triggers active circuit breaker or contactor shutdowns. It only displays recommended actions on the dashboard.
- * **Advanced Filtering / Profiling:** Do not claim that the gateway performs advanced noise filtering or multi-sensor spatial thermal profiling. These features are not implemented.